

Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

Лабораторная работа №4

Классический криптографический алгоритм DES

Выполнил: Тихонов А. В.
студент группы ИВТ-42-23
Проверил:
Ковалев. С. В.

Чебоксары, 2026

Цель работы

Целью работы является знакомство с классическим криптографическим алгоритмом DES и его работой в режиме ECB.

Индивидуальное задание.

В данной лабораторной работе необходимо программно реализовать алгоритмы шифрования и дешифрования DES в режиме ECB. Различные индивидуальные задания не предусматриваются. Язык программирования студент выбирает самостоятельно.

Функция шифрования.

1. Подготовка текста (Padding)

Если ваш текст не делится ровно на блоки по 8 байт, в конец добавляются недостающие байты). К примеру, если не хватает 3 байт, дописываются три байта со значением 0x03.

2. Генерация 16 «подключей»

Из вашего одного исходного ключа создаются **16 разных мини-ключей** (по одному на каждый раунд шифрования). Для этого ключ делят пополам, циклически сдвигают биты в каждой половине на каждом шаге и выбирают из них нужные биты с помощью специальных таблиц-перестановок (PC1 и PC2).

3. Начальная перестановка (IP)

8 байт входящего блока данных перемешиваются по строго заданной таблице (IP).

4. Основной цикл: 16 раундов (Сеть Фейстеля)

Блок из 64 бит делится на две равные части: **Левую (L)** и **Правую (R)** по 32 бита.

В каждом из 16 раундов с правой половиной (R) происходит магия:

1. **Расширение (E):** 32 бита превращаются в 48 бит путем дублирования некоторых бит.
2. **Сложение с ключом (XOR):** Эти 48 бит складываются с мини-ключом текущего раунда.
3. **S-блоки (Замена):** 48 бит разбиваются на 8 групп по 6 бит. Каждая группа отправляется в свою табличку-матрицу (**S-блок**) и превращается в 4 бита.
4. **Перестановка (P):** Полученные 32 бита снова перемешиваются.
5. **Сложение и сдвиг:** Результат складывается (XOR) с **Левой половиной (L)**. После этого Левая и Правая половины **меняются местами** и переходят в следующий раунд.

5. Финальная перестановка (IP⁻¹)

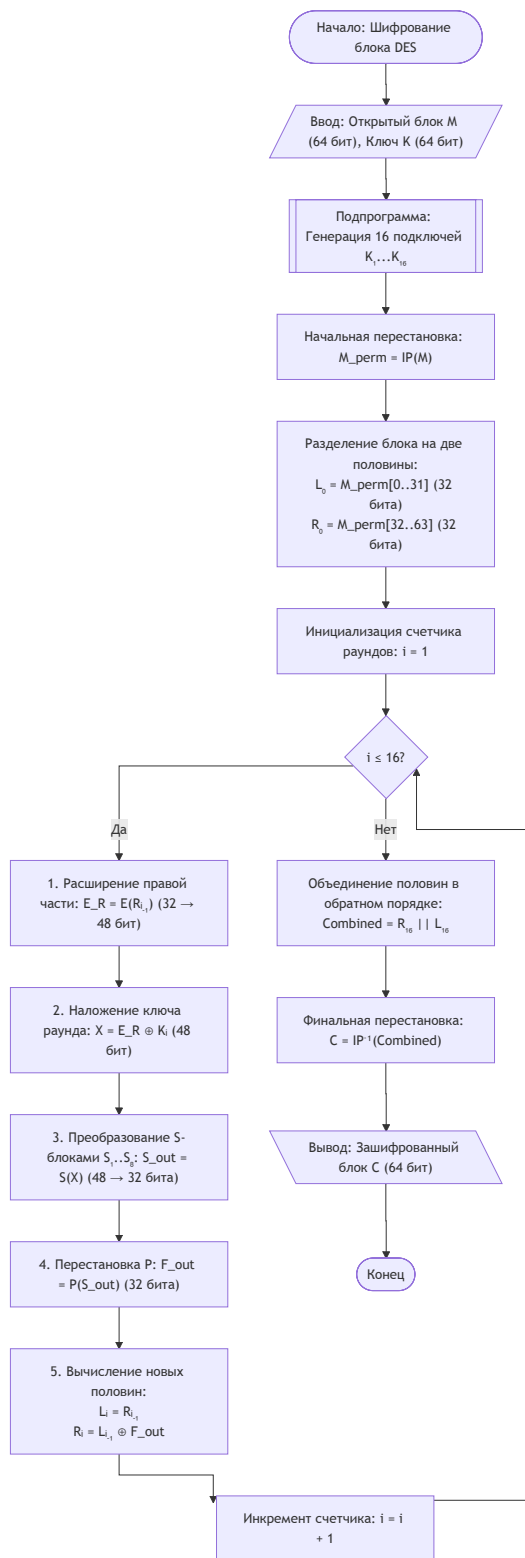
После 16 раундов левая и правая половинки соединяются вместе и биты перемешиваются в последний раз (операцией, обратной начальной перестановке). На выходе получается зашифрованный блок.

Как происходит расшифровка?

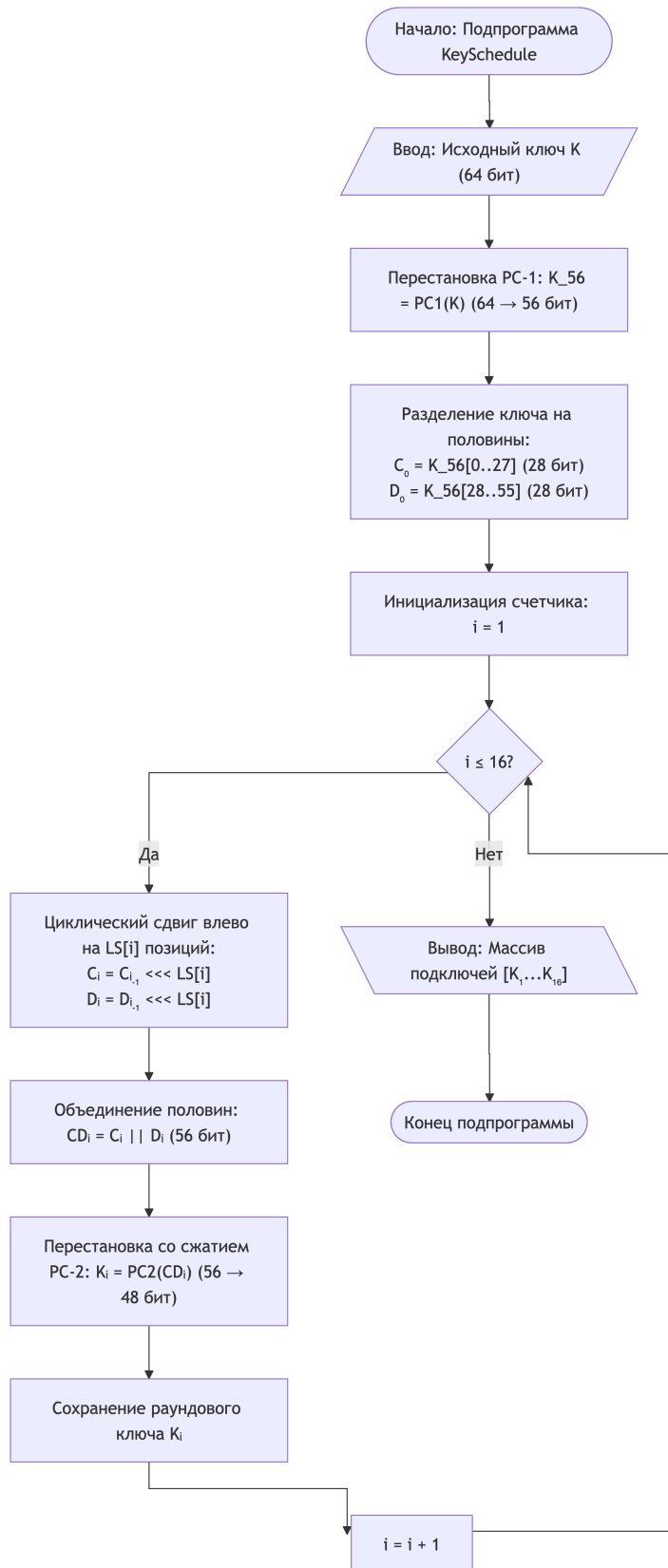
Алгоритм расшифровки **абсолютно идентичен** шифрованию. Единственное отличие — 16 мини-ключей применяются в **обратном порядке** (от 16-го к 1-му).

ГСА

Основная программа



Подпрограмма генерации ключей



```

package main

import (
    "bufio"
    "errors"
    "fmt"
    "os"
    "strings"
)

type BitArray struct {
    bits []bool
}

func NewBitArray(size int) *BitArray {
    return &BitArray{bits: make([]bool, size)}
}

func NewBitArrayFromHex(hexStr string) *BitArray {
    ba := &BitArray{}
    ba.FromHex(hexStr)
    return ba
}

func (ba *BitArray) Size() int {
    return len(ba.bits)
}

func (ba *BitArray) Get(index int) bool {
    if index >= len(ba.bits) {
        return false
    }
    return ba.bits[index]
}

func (ba *BitArray) Set(index int, value bool) {
    if index < len(ba.bits) {
        ba.bits[index] = value
    }
}

func (ba *BitArray) Flip(index int) {
    if index < len(ba.bits) {
        ba.bits[index] = !ba.bits[index]
    }
}

```

```
}
```

```
func (ba *BitArray) XOR(other *BitArray) *BitArray {  
    maxSize := len(ba.bits)  
    if len(other.bits) > maxSize {  
        maxSize = len(other.bits)  
    }  
    result := NewBitArray(maxSize)  
    for i := 0; i < maxSize; i++ {  
        result.Set(i, ba.Get(i) != other.Get(i))  
    }  
    return result  
}
```

```
func (ba *BitArray) Substring(start, length int) *BitArray {  
    result := NewBitArray(length)  
    for i := 0; i < length && (start+i) < len(ba.bits); i++ {  
        result.Set(i, ba.bits[start+i])  
    }  
    return result  
}
```

```
func (ba *BitArray) Concatenate(other *BitArray) {  
    ba.bits = append(ba.bits, other.bits...)  
}
```

```
func (ba *BitArray) CyclicLeftShift(positions int) {  
    if len(ba.bits) == 0 {  
        return  
    }  
    positions %= len(ba.bits)  
    newBits := make([]bool, len(ba.bits))  
    for i := 0; i < len(ba.bits); i++ {  
        newBits[i] = ba.bits[(i+positions)%len(ba.bits)]  
    }  
    ba.bits = newBits  
}
```

```
func (ba *BitArray) FromHex(hexStr string) {  
    ba.bits = make([]bool, 0, len(hexStr)*4)  
    for _, c := range hexStr {  
        var value int  
        if c >= '0' && c <= '9' {  
            value = int(c - '0')  
        } else if c >= 'A' && c <= 'F' {  
            value = int(c - 'A' + 10)  
        }  
    }  
}
```

```

    } else if c >= 'a' && c <= 'f' {
        value = int(c - 'a' + 10)
    }

    for i := 3; i >= 0; i-- {
        ba.bits = append(ba.bits, ((value>>i)&1) == 1)
    }
}

func (ba *BitArray) ToHex() string {
    var sb strings.Builder
    for i := 0; i < len(ba.bits); i += 4 {
        value := 0
        for j := 0; j < 4 && (i+j) < len(ba.bits); j++ {
            bitVal := 0
            if ba.bits[i+j] {
                bitVal = 1
            }
            value = (value << 1) | bitVal
        }
        sb.WriteString(fmt.Sprintf("%X", value))
    }
    return sb.String()
}

func (ba *BitArray) ToBinary() string {
    var sb strings.Builder
    for _, bit := range ba.bits {
        if bit {
            sb.WriteRune('1')
        } else {
            sb.WriteRune('0')
        }
    }
    return sb.String()
}

func (ba *BitArray) Print() {
    fmt.Println(ba.ToBinary())
}

// =====
// Скрытка DES
// =====

```

```
var (  
  IP = [64]int{  
    58, 50, 42, 34, 26, 18, 10, 2,  
    60, 52, 44, 36, 28, 20, 12, 4,  
    62, 54, 46, 38, 30, 22, 14, 6,  
    64, 56, 48, 40, 32, 24, 16, 8,  
    57, 49, 41, 33, 25, 17, 9, 1,  
    59, 51, 43, 35, 27, 19, 11, 3,  
    61, 53, 45, 37, 29, 21, 13, 5,  
    63, 55, 47, 39, 31, 23, 15, 7,  
  }
```

```
  IP_1 = [64]int{  
    40, 8, 48, 16, 56, 24, 64, 32,  
    39, 7, 47, 15, 55, 23, 63, 31,  
    38, 6, 46, 14, 54, 22, 62, 30,  
    37, 5, 45, 13, 53, 21, 61, 29,  
    36, 4, 44, 12, 52, 20, 60, 28,  
    35, 3, 43, 11, 51, 19, 59, 27,  
    34, 2, 42, 10, 50, 18, 58, 26,  
    33, 1, 41, 9, 49, 17, 57, 25,  
  }
```

```
  E = [48]int{  
    32, 1, 2, 3, 4, 5,  
    4, 5, 6, 7, 8, 9,  
    8, 9, 10, 11, 12, 13,  
    12, 13, 14, 15, 16, 17,  
    16, 17, 18, 19, 20, 21,  
    20, 21, 22, 23, 24, 25,  
    24, 25, 26, 27, 28, 29,  
    28, 29, 30, 31, 32, 1,  
  }
```

```
  P = [32]int{  
    16, 7, 20, 21, 29, 12, 28, 17,  
    1, 15, 23, 26, 5, 18, 31, 10,  
    2, 8, 24, 14, 32, 27, 3, 9,  
    19, 13, 30, 6, 22, 11, 4, 25,  
  }
```

```
  PC1 = [56]int{  
    57, 49, 41, 33, 25, 17, 9,  
    1, 58, 50, 42, 34, 26, 18,  
    10, 2, 59, 51, 43, 35, 27,  
    19, 11, 3, 60, 52, 44, 36,
```

```

63, 55, 47, 39, 31, 23, 15,
7, 62, 54, 46, 38, 30, 22,
14, 6, 61, 53, 45, 37, 29,
21, 13, 5, 28, 20, 12, 4,
}

```

```

PC2 = [48]int{
14, 17, 11, 24, 1, 5,
3, 28, 15, 6, 21, 10,
23, 19, 12, 4, 26, 8,
16, 7, 27, 20, 13, 2,
41, 52, 31, 37, 47, 55,
30, 40, 51, 45, 33, 48,
44, 49, 39, 56, 34, 53,
46, 42, 50, 36, 29, 32,
}

```

```

LS = [16]int{1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1}

```

```

S = [8][4][16]int{
// S1
{
{14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7},
{0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8},
{4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0},
{15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13},
},
// S2
{
{15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10},
{3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5},
{0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15},
{13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9},
},
// S3
{
{10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8},
{13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1},
{13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7},
{1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12},
},
// S4
{
{7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15},
{13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9},
{10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4},
}
}

```

```

        {3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14},
    },
    // S5
    {
        {2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9},
        {14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6},
        {4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14},
        {11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3},
    },
    // S6
    {
        {12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11},
        {10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8},
        {9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6},
        {4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13},
    },
    // S7
    {
        {4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1},
        {13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6},
        {1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2},
        {6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12},
    },
    // S8
    {
        {13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7},
        {1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2},
        {7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 13, 5, 8},
        {2, 1, 14, 7, 4, 10, 8, 13, 5, 12, 9, 0, 3, 5, 5, 11},
    },
    },
}
)

type DES struct {
    subkeys []*BitArray
}

func NewDES() *DES {
    return &DES{
        subkeys: make([]*BitArray, 0),
    }
}

// Применение перестановки
func (d *DES) applyPermutation(input *BitArray, table []int, outputSize int) *BitArray {
    output := NewBitArray(outputSize)

```

```

    for i := 0; i < outputSize; i++ {
        output.Set(i, input.Get(table[i]-1))
    }
    return output
}

// Функция f(R, K)
func (d *DES) f(R, K *BitArray) *BitArray {
    // Расширение E
    expanded := d.applyPermutation(R, E[:], 48)

    // XOR с ключом
    xored := expanded.XOR(K)

    // S-блоки
    sboxOutput := NewBitArray(32)
    for i := 0; i < 8; i++ {
        block := xored.Substring(i*6, 6)

        // Строка: первый и последний биты
        row := 0
        if block.Get(0) {
            row |= 2
        }
        if block.Get(5) {
            row |= 1
        }

        // Столбец: средние 4 бита
        col := 0
        for j := 1; j <= 4; j++ {
            col = (col << 1)
            if block.Get(j) {
                col |= 1
            }
        }

        value := S[i][row][col]

        // Преобразование в 4 бита
        for j := 3; j >= 0; j-- {
            sboxOutput.Set(i*4+(3-j), ((value>>j)&1) == 1)
        }
    }

    // Перестановка P

```

```

    return d.applyPermutation(sboxOutput, P[:], 32)
}

// Генерация подключей
func (d *DES) generateSubkeys(key *BitArray) {
    d.subkeys = make([]*BitArray, 0, 16)

    // Перестановка PC1
    permutedKey := d.applyPermutation(key, PC1[:], 56)

    // Разделение на L и R
    L := permutedKey.Substring(0, 28)
    R := permutedKey.Substring(28, 56)

    // 16 раундов
    for i := 0; i < 16; i++ {
        // Циклический сдвиг
        L.CyclicLeftShift(LS[i])
        R.CyclicLeftShift(LS[i])

        // Объединение
        combined := NewBitArray(56)
        for j := 0; j < L.Size(); j++ {
            combined.Set(j, L.Get(j))
        }
        for j := 0; j < R.Size(); j++ {
            combined.Set(L.Size()+j, R.Get(j))
        }

        // Перестановка PC2
        subkey := d.applyPermutation(combined, PC2[:], 48)
        d.subkeys = append(d.subkeys, subkey)
    }
}

// Шифрование одного блока
func (d *DES) encryptBlock(block *BitArray, decrypt bool) *BitArray {
    // Начальная перестановка IP
    permuted := d.applyPermutation(block, IP[:], 64)

    // Разделение на L и R
    L := permuted.Substring(0, 32)
    R := permuted.Substring(32, 64)

    // 16 раундов
    for i := 0; i < 16; i++ {

```

```

    keyIndex := i
    if decrypt {
        keyIndex = 15 - i
    }
    tempL := L
    L = R
    R = tempL.XOR(d.f(R, d.subkeys[keyIndex]))
}

// Объединение R + L (обратный порядок!)
combined := NewBitArray(64)
for j := 0; j < R.Size(); j++ {
    combined.Set(j, R.Get(j))
}
for j := 0; j < L.Size(); j++ {
    combined.Set(R.Size()+j, L.Get(j))
}

// Финальная перестановка IP^-1
return d.applyPermutation(combined, IP_1[:], 64)
}

// Установка ключа
func (d *DES) SetKey(hexKey string) {
    key := NewBitArrayFromHex(hexKey)
    d.generateSubkeys(key)
}

// Шифрование в режиме ECB
func (d *DES) Encrypt(hexPlaintext string) string {
    plaintext := NewBitArrayFromHex(hexPlaintext)
    ciphertext := d.encryptBlock(plaintext, false)
    return ciphertext.ToHex()
}

// Дешифрование в режиме ECB
func (d *DES) Decrypt(hexCiphertext string) string {
    ciphertext := NewBitArrayFromHex(hexCiphertext)
    plaintext := d.encryptBlock(ciphertext, true)
    return plaintext.ToHex()
}

// Изменение одного бита в шифротексте
func (d *DES) FlipBit(hexCiphertext string, bitPosition int) string {
    ciphertext := NewBitArrayFromHex(hexCiphertext)
    ciphertext.Flip(bitPosition)
}

```

```

    return ciphertext.ToHex()
}

// =====
// Вспомогательные функции
// =====

// Функция, переводящая text в hex
func stringToHex(input string) string {
    const hexUpper = "0123456789ABCDEF"
    var sb strings.Builder
    sb.Grow(len(input) * 2)

    for i := 0; i < len(input); i++ {
        c := input[i]
        sb.WriteByte(hexUpper[c>>4]) // Старший (4 бита)
        sb.WriteByte(hexUpper[c&0x0F]) // Младший (4 бита)
    }

    return sb.String()
}

// Выравнивание текста до кратности blockSize (по умолчанию 8)
func padPKCS7(input string, blockSize int) string {
    paddingLen := blockSize - (len(input) % blockSize)
    padding := strings.Repeat(string(rune(paddingLen)), paddingLen)
    return input + padding
}

// Восстановление исходного текста (удаление наполнителя)
func unpadPKCS7(input string, blockSize int) (string, error) {
    if len(input) == 0 || len(input)%blockSize != 0 {
        return "", errors.New("длина данных не кратна размеру блока")
    }

    padLen := int(input[len(input)-1])

    if padLen == 0 || padLen > blockSize || padLen > len(input) {
        return "", errors.New("некорректный формат PKCS#7 паддинга")
    }

    // Проверка целостности: все padLen байт должны быть равны padLen
    for i := len(input) - padLen; i < len(input); i++ {
        if int(input[i]) != padLen {
            return "", errors.New("недействительный PKCS#7 паддинг (байты не совпадают)")
        }
    }
}

```

```

    }

    return input[:len(input)-padLen], nil
}

// =====
// Демонстрация работы DES
// =====

type DESDemo struct {
    des *DES
}

func NewDESDemo() *DESDemo {
    return &DESDemo{des: NewDES()}
}

func (demo *DESDemo) Run() {
    scanner := bufio.NewScanner(os.Stdin)

    // Тест 1: Базовое шифрование/дешифрование
    fmt.Println("DES шифрование и дешифрование:")
    fmt.Println()
    fmt.Println()

    key := "133457799BBCDFF1"
    fmt.Println("Введите открытый текст")

    rawPlaintext := ""
    if scanner.Scan() {
        rawPlaintext = scanner.Text()
    }

    fmt.Println("Ключ:", key)
    fmt.Println("Открытый текст:", rawPlaintext)
    demo.des.SetKey(key)

    paddedPlaintext := padPKCS7(rawPlaintext, 8)
    fullPlaintextHex := ""
    fullCiphertextHex := ""
    fullDecryptedHex := ""

    fmt.Println("Поблочная обработка (размер блока: 8 байт / 64 бита)")
    fmt.Println()

    // 2. Цикл разделения на блоки по 8 символов/байт

```

```

for i := 0; i < len(paddedPlaintext); i += 8 {
    // Извлекаем блок в 8 байт
    block := paddedPlaintext[i : i+8]

    // Переводим 8-байтовый блок в HEX (16 символов HEX)
    blockHex := stringToHex(block)
    fullPlaintextHex += blockHex

    // Шифруем и дешифруем конкретный блок
    blockCipherHex := demo.des.Encrypt(blockHex)
    blockDecryptedHex := demo.des.Decrypt(blockCipherHex)

    fullCiphertextHex += blockCipherHex
    fullDecryptedHex += blockDecryptedHex

    matchStr := "НЕТ"
    if blockHex == blockDecryptedHex {
        matchStr = "ДА"
    }

    fmt.Printf("Блок %d:\n", (i/8)+1)
    fmt.Printf(" Символы (8 байт):  \">%s\<"\n", block)
    fmt.Printf(" Открытый (HEX):  %s\n", blockHex)
    fmt.Printf(" Шифротекст (HEX):  %s\n", blockCipherHex)
    fmt.Printf(" Дешифрован (HEX):  %s\n", blockDecryptedHex)
    fmt.Printf(" Совпадение блока:  %s\n", matchStr)
    fmt.Println("-----")
}

fmt.Println()
fmt.Println("Итоговый результат")
fmt.Println("открытый текст (HEX):", fullPlaintextHex)
fmt.Println("Полный шифротекст (HEX):", fullCiphertextHex)
fmt.Println("Расшифрованный текст:  ", fullDecryptedHex)

fullMatchStr := "НЕТ"
if fullPlaintextHex == fullDecryptedHex {
    fullMatchStr = "ДА"
}
fmt.Println("Совпадает с оригиналом: ", fullMatchStr)

// Тест 2: Размножение ошибок
fmt.Println("2. Демонстрация размножения ошибок (ECB) на первом блоке:")
fmt.Println()
fmt.Println()

```

```

samplePlainText := fullPlaintextHex[:16]
sampleCipherText := fullCiphertextHex[:16]
fmt.Println("Оригинальный шифротекст:", sampleCipherText)

// Изменяем 10-й бит (считаем с 0)
modifiedCipherText := demo.des.FlipBit(sampleCipherText, 10)
fmt.Println("Шифротекст с измененным 10-м битом:", modifiedCipherText)

modifiedDecrypted := demo.des.Decrypt(modifiedCipherText)
fmt.Println("Дешифрованный текст:", modifiedDecrypted)
fmt.Println("Оригинальный текст:", samplePlainText)
fmt.Print("Различий в битах: ")

orig := NewBitArrayFromHex(samplePlainText)
modified := NewBitArrayFromHex(modifiedDecrypted)
diffCount := 0
for i := 0; i < orig.Size(); i++ {
    if orig.Get(i) != modified.Get(i) {
        diffCount++
    }
}
fmt.Printf("%d из %d\n\n", diffCount, orig.Size())

// Тест 3: Слабые ключи
fmt.Println("3. Работа со слабыми ключами:")
fmt.Println()
fmt.Println()

weakKeys := []string{
    "0101010101010101",
    "FEFEFEFEFEFEFEFE",
    "1F1F1F1F0E0E0E0E",
    "E0E0E0E0F1F1F1F1",
}

for _, weakKey := range weakKeys {
    fmt.Println("Слабый ключ:", weakKey)
    demo.des.SetKey(weakKey)

    ct1 := demo.des.Encrypt(samplePlainText)
    fmt.Println(" Первое шифрование:", ct1)

    ct2 := demo.des.Encrypt(ct1)
    fmt.Println(" Второе шифрование:", ct2)

    returnMatchStr := "HET"

```

```
        if samplePlainText == ct2 {
            returnMatchStr = "ДА"
        }
        fmt.Println(" Возврат к оригиналу:", returnMatchStr)
        fmt.Println()
    }
}

func main() {
    demo := NewDESDemo()
    demo.Run()
}
```

Запуск программы

```
goose — -zsh — 75x28

DES шифрование и дешифрование:

Введите открытый текст
heller dramatischer Himmel
Ключ: 133457799BBCDF1
Открытый текст: heller dramatischer Himmel
Поблочная обработка (размер блока: 8 байт / 64 бита)

Блок 1:
Символы (8 байт): "heller d"
Открытый (HEX): 68656C6C65722064
Шифротекст (HEX): CFB2A9E964566390
Дешифрован (HEX): 68656C6C65722064
Совпадение блока: ДА
-----
Блок 2:
Символы (8 байт): "ramatisc"
Открытый (HEX): 72616D6174697363
Шифротекст (HEX): 2B490ED9A8E38B3B
Дешифрован (HEX): 72616D6174697363
Совпадение блока: ДА
-----
Блок 3:
Символы (8 байт): "her Himm"
Открытый (HEX): 6865722048696D6D
Шифротекст (HEX): 04ECBAB1CB0B7FA6
Дешифрован (HEX): 6865722048696D6D

Дешифрован (HEX): 72616D6174697363
Совпадение блока: ДА
-----
Блок 3:
Символы (8 байт): "her Himm"
Открытый (HEX): 6865722048696D6D
Шифротекст (HEX): 04ECBAB1CB0B7FA6
Дешифрован (HEX): 6865722048696D6D
Совпадение блока: ДА
-----
Блок 4:
Символы (8 байт): "e1"
Открытый (HEX): 656C060606060606
Шифротекст (HEX): B073E24011F49773
Дешифрован (HEX): 656C060606060606
Совпадение блока: ДА
-----

Итоговый результат
открытый текст (HEX): 68656C6C6572206472616D61746973636865722048696D6D656C0
60606060606
Полный шифротекст (HEX): CFB2A9E9645663902B490ED9A8E38B3B04ECBAB1CB0B7FA6B0
73E24011F49773
Расшифрованный текст: 68656C6C6572206472616D61746973636865722048696D6D65
6C060606060606
Совпадает с оригиналом: ДА
2. Демонстрация размножения ошибок (ЕСВ) на первом блоке:

БЛОК 3:
```

```
DES шифрование и дешифрование:
goose — -zsh — 75x28

Оригинальный шифротекст: CF92A9E964566390
Шифротекст с измененным 10-м битом: CF92A9E964566390
Дешифрованный текст: 6E0401057BF854A7
Оригинальный текст: 68656C6C65722064
Различий в битах: 29 из 64

3. Работа со слабыми ключами:

Слабый ключ: 0101010101010101
Первое шифрование: C4EBB435146F6F47
Второе шифрование: 68656C6C65722064
Возврат к оригиналу: ДА

Слабый ключ: FEFEFEFEFEFEFEFE
Первое шифрование: 83D98BB2373F5E90
Второе шифрование: 68656C6C65722064
Возврат к оригиналу: ДА

Слабый ключ: 1F1F1F1F0E0E0E0E
Первое шифрование: 023F8024572C941C
Второе шифрование: 68656C6C65722064
Возврат к оригиналу: ДА

Слабый ключ: E0E0E0E0F1F1F1F1
Первое шифрование: C28B0D1358FF9EE2
Второе шифрование: 68656C6C65722064
Возврат к оригиналу: ДА
```